MOHAN KTK • YOUTUBE LIVE • MYSQL TUTORIAL

SQL Complete Topics

All 13 Sections — Beginner to Interview-Ready

A structured MySQL walkthrough covering every essential topic. Each section includes concept explanation, live syntax, and real-world examples — designed for Telugu YouTube live class students.

13	60	120+
Sections	Slides	MySQL Queries

Complete SQL Syllabus

1. **Basic SQL Queries**
   SELECT, DISTINCT, WHERE, ORDER BY
2. **Operators**
   AND/OR, NOT, BETWEEN, IN, LIKE
3. **DML Operations**
   INSERT, UPDATE, DELETE, Copy Table
4. **Table & DB Concepts**
   CREATE TABLE, Data Types, Temp Tables
5. **NULL Handling**
   IS NULL, IS NOT NULL, COALESCE
6. **Aggregate Functions**
   MAX, MIN, SUM, AVG, COUNT
7. **GROUP BY & Filtering**
   GROUP BY, HAVING, WHERE vs HAVING
8. **Joins**
   INNER, LEFT, RIGHT, FULL, SELF, ANTI
9. **Set Operations**
   UNION, UNION ALL
10. **Conditional Logic**
    CASE, Nested CASE
11. **Advanced SQL**
    Subqueries, Execution Order, TOP N
12. **Constraints**
    NOT NULL, UNIQUE, CHECK, DEFAULT, PK, FK
13. **Deletion Operations**
    DELETE, DROP, TRUNCATE

Basic SQL Queries

SELECT • DISTINCT • WHERE • ORDER BY

SQL SELECT Statement

Used to retrieve data from one or more tables — the most-used SQL command.

SYNTAX

```
SELECT  column1, column2  FROM  table_name;          -- Specific columns
SELECT  *                 FROM  table_name;          -- All columns
```

EXAMPLES

```
-- Get all employees
SELECT * FROM employees;

-- Get specific columns
SELECT name, salary
FROM   employees;

-- Column alias with AS
SELECT name, salary AS monthly_pay
FROM   employees;

-- Multiple aliases
SELECT emp_id    AS id,
       name      AS employee_name,
       salary    AS pay
FROM   employees;
```

📋 RESULT — employees

emp_id	name	salary
101	Arjun Kumar	65,000
102	Priya Sharma	85,000
103	Ravi Mehta	72,000
104	Sneha Reddy	90,000

💡 **BEST PRACTICE:**

SELECT * is fine for exploration but AVOID in production — always specify columns. It wastes network bandwidth and makes code fragile when table structure changes.

SELECT DISTINCT

Returns only unique values — removes duplicate rows from the result set.

SYNTAX & USAGE

```
-- All cities (includes duplicates)
SELECT city FROM employees;

-- Only unique cities
SELECT DISTINCT city
FROM   employees;

-- Count of unique values
SELECT COUNT(DISTINCT city) AS unique_cities
FROM   employees;

-- Distinct on multiple columns
SELECT DISTINCT city, department
FROM   employees;
```

WITHOUT DISTINCT

city
Hyderabad
Bengaluru
Hyderabad
Mumbai
Bengaluru
Hyderabad

WITH DISTINCT

city
Hyderabad
Bengaluru
Mumbai

💡 **WHEN TO USE DISTINCT:**

Find all unique categories in a column • Count how many different values exist • Remove duplicate entries from reports

SQL WHERE Clause

Filters rows — returns only records that satisfy the specified condition.

```
SELECT columns FROM table WHERE condition;
```

EXAMPLES

```
-- Number comparison
SELECT * FROM employees
WHERE  salary > 50000;

-- Text match (case-sensitive in MySQL)
SELECT * FROM employees
WHERE  city = 'Hyderabad';

-- Multiple conditions with AND
SELECT * FROM employees
WHERE  salary > 50000
  AND  city = 'Bengaluru';

-- Date filter
SELECT * FROM orders
WHERE  order_date > '2024-01-01';

-- Not equal
SELECT * FROM employees
WHERE  department != 'HR';
```

Operator	Meaning	Example
=	Equal to	WHERE city = 'Mumbai'
!= or <>	Not equal	WHERE dept != 'HR'
> or <	Greater / Less	WHERE salary > 50000
>= or <=	≥ or ≤	WHERE age >= 25
BETWEEN	Within range	BETWEEN 30000 AND 80000
LIKE	Pattern match	WHERE name LIKE 'A%'
IN	In a list	WHERE city IN ('Pune','Goa')
IS NULL	Null check	WHERE bonus IS NULL

SQL ORDER BY Clause

Sorts the result set by one or more columns in ASC (default) or DESC order.

```
SELECT columns FROM table [WHERE ...] ORDER BY col1 [ASC|DESC], col2 [ASC|DESC];
```

EXAMPLES

```
-- Default ASC (low → high)
SELECT * FROM employees
ORDER BY salary;

-- DESC – top earners first
SELECT * FROM employees
ORDER BY salary DESC;

-- Alphabetical by name
SELECT * FROM employees
ORDER BY name ASC;

-- Multiple columns
SELECT * FROM employees
ORDER BY department ASC,
         salary DESC;

-- ORDER BY with WHERE
SELECT * FROM employees
WHERE  city = 'Mumbai'
ORDER BY salary DESC
LIMIT  5;
```

ASC — Ascending (Default)

Smallest → Largest • A → Z • Oldest → Newest
No need to write ASC — it is the default behaviour.

DESC — Descending

Largest → Smallest • Z → A • Newest → Oldest
MUST be written explicitly — not the default.

💡 **REMEMBER:**

ORDER BY is always the LAST clause (except LIMIT). You can ORDER BY a column even if it's not in the SELECT list.

Operators in SQL

AND/OR • NOT • BETWEEN • IN • LIKE

AND • OR • NOT Operators

Combine or negate conditions in WHERE clause — logical operators.

AND

ALL conditions must be TRUE

```
SELECT * FROM employees
WHERE  salary > 50000
  AND  city = 'Bengaluru';

-- Both must be true
-- Returns Bengaluru
-- employees earning > 50k
```

OR

ANY condition must be TRUE

```
SELECT * FROM employees
WHERE  city = 'Mumbai'
    OR city = 'Pune';

-- Either can be true
-- Returns employees from
-- Mumbai OR Pune (or both)
```

NOT

Reverses / negates condition

```
SELECT * FROM employees
WHERE  NOT department = 'HR';

-- Equivalent to:
WHERE  department != 'HR'

-- NOT with IN
WHERE  city NOT IN ('Delhi','Remote')
```

BETWEEN & IN Operators

BETWEEN

Selects values within a given range. INCLUSIVE — both end values are included.

BETWEEN

```
-- Number range
SELECT * FROM employees
WHERE  salary BETWEEN 40000 AND 80000;

-- Date range
SELECT * FROM orders
WHERE  order_date
       BETWEEN '2024-01-01'
           AND '2024-12-31';

-- Text range (alphabetical)
SELECT * FROM employees
WHERE  name BETWEEN 'A' AND 'M';

-- NOT BETWEEN
WHERE  salary NOT BETWEEN 40000 AND 80000;
```

IN

Checks if a value matches any value in a list. Shorthand for multiple OR conditions.

IN

```
-- Value in a list
SELECT * FROM employees
WHERE  city IN ('Mumbai','Pune','Goa');

-- NOT IN
SELECT * FROM employees
WHERE  department
       NOT IN ('HR', 'Legal', 'Admin');

-- IN with subquery
SELECT * FROM employees
WHERE  dept_id IN (
  SELECT id FROM departments
  WHERE  location = 'South'
);
```

LIKE Operator — Pattern Matching

Search for a pattern in a column using wildcard characters. Used with WHERE.

WILDCARDS

% — **Zero or more characters**

`'A%'` → Arjun, Akash, Anita, Anu

_ — **Exactly ONE character**

`'R__i'` → Ravi (4 chars, starts R, ends i)

LIKE EXAMPLES

```
-- Starts with 'A'
SELECT * FROM employees
WHERE  name LIKE 'A%';

-- Ends with 'ar'
WHERE  name LIKE '%ar';

-- Contains 'kumar' anywhere
WHERE  name LIKE '%kumar%';

-- Second char is 'a'
WHERE  name LIKE '_a%';

-- Exactly 5 characters
WHERE  name LIKE '_____';

-- NOT LIKE
WHERE  email NOT LIKE '%gmail%';
```

Pattern	Matches
`'A%'`	Arjun, Akash, Aisha, Anu…
`'%ar'`	Kumar, Sagar, Anwar…
`'%ma%'`	Priyamala, Manoj, Rama…
`'_a%'`	Ravi, Navin, Sarita…
`'K_mar'`	Kumar (exactly 5 chars)
`'2024%'`	Any string starting with 2024
`'%@gmail%'`	Email containing @gmail

DML Operations

INSERT • UPDATE • DELETE • Copy Table

INSERT INTO Statement

Adds new rows of data into a table. Two methods — all columns or specific columns.

Method 1: All Columns

Provide values for ALL columns in the exact order they appear in the table.

ALL COLUMNS

```
-- Single row insert
INSERT INTO employees
VALUES (105, 'Vikram', 'Delhi', 68000);

-- Multiple rows at once (faster!)
INSERT INTO employees
VALUES
  (106, 'Anita',  'Chennai', 72000),
  (107, 'Kiran',  'Pune',    65000),
  (108, 'Suresh', 'Kochi',   58000);
```

Method 2: Specific Columns

Specify column names — skipped columns get NULL or their DEFAULT value.

SPECIFIC COLUMNS

```
-- Only specified columns
INSERT INTO employees
  (emp_id, name, salary)
VALUES
  (109, 'Meena', 75000);
-- city gets NULL above

-- Insert from another table
INSERT INTO emp_backup
SELECT * FROM employees
WHERE  salary > 80000;

-- Insert with subquery
INSERT INTO dept_summary (dept, avg_sal)
SELECT department, AVG(salary)
FROM   employees
GROUP BY department;
```

UPDATE Statement

Modifies existing data in a table. ALWAYS use WHERE — without it you update EVERY row!

```
UPDATE table_name SET col1=val1, col2=val2 WHERE condition;
```

UPDATE EXAMPLES

```
-- Update single column
UPDATE employees
SET    salary = 70000
WHERE  emp_id = 101;

-- Update multiple columns
UPDATE employees
SET    salary     = 90000,
       department = 'Senior Analytics',
       city       = 'Bengaluru'
WHERE  emp_id = 102;

-- Update based on condition
UPDATE employees
SET    salary = salary * 1.15
WHERE  rating = 'Excellent';

-- Update with calculation
UPDATE orders
SET    total = quantity * unit_price
WHERE  total IS NULL;
```

⚠️ DANGER — MISSING WHERE

If you forget WHERE, the UPDATE affects EVERY single row in the table:

DANGER vs SAFE

```
-- ❌ DISASTER: No WHERE
UPDATE employees
SET    salary = 0;
-- Sets ALL 10,000 employees to ₹0!

-- ✅ SAFE: Always use WHERE
UPDATE employees
SET    salary = 0
WHERE  emp_id = 9999;
```

💡 SAFE UPDATE MODE:

In MySQL Workbench, Safe Update Mode prevents UPDATE/DELETE without WHERE. Enable via Edit → Preferences → SQL Editor.

DELETE Statement

Removes specific rows (or all rows) from a table. Can be rolled back inside a transaction.

```
DELETE FROM table_name [WHERE condition];  -- WHERE is optional but CRITICAL!
```

DELETE EXAMPLES

```
-- Delete one specific row
DELETE FROM employees
WHERE  emp_id = 101;

-- Delete with condition
DELETE FROM orders
WHERE  status = 'Cancelled'
  AND  order_date < '2023-01-01';

-- Delete using subquery
DELETE FROM employees
WHERE  dept_id NOT IN (
  SELECT id FROM departments
);

-- Delete all rows (recoverable)
DELETE FROM temp_data;
-- Same as TRUNCATE but slower
-- CAN be rolled back
```

	DELETE	TRUNCATE	DROP
Removes	Specific/all rows	ALL rows	Entire table
Structure	Kept	Kept	Gone
WHERE	✅ Yes	❌ No	❌ No
Rollback	✅ Yes	❌ No	❌ No
Speed	Slow	Fast	Instant
AUTO_INC	Not reset	Reset	N/A

💡 **KEY RULE:**

DELETE is DML — it logs every deleted row, supports WHERE, and CAN be rolled back. TRUNCATE is DDL — it's faster but cannot be rolled back.

Copying Data from One Table to Another

MySQL methods to duplicate, back up, or transfer data between tables.

COPY METHODS

```
-- Method 1: CREATE TABLE AS SELECT
-- (creates new table AND copies data)
CREATE TABLE emp_backup
AS SELECT * FROM employees;

-- Copy only selected columns
CREATE TABLE emp_summary
AS SELECT emp_id, name, salary
   FROM   employees;

-- Copy with filter
CREATE TABLE high_earners
AS SELECT * FROM employees
   WHERE  salary > 80000;

-- Copy to EXISTING table
INSERT INTO emp_archive
SELECT * FROM employees
WHERE  status = 'Inactive';

-- Copy structure only (no data)
CREATE TABLE emp_template
LIKE employees;
```

VERIFY COPY

```
-- SELECT INTO (SQL Server syntax —
-- NOT supported in MySQL):
-- SELECT * INTO new_tbl FROM old_tbl

-- MySQL equivalent:
CREATE TABLE new_table
AS SELECT * FROM old_table;

-- Verify the copy
SELECT COUNT(*) FROM emp_backup;

-- Compare row counts
SELECT 'Original'   AS src,
       COUNT(*)     AS rows
FROM   employees
UNION ALL
SELECT 'Backup'     AS src,
       COUNT(*)     AS rows
FROM   emp_backup;
```

💡 **MYSQL NOTE:**

MySQL does NOT support SELECT INTO (that's SQL Server syntax). Always use CREATE TABLE ... AS SELECT or INSERT INTO ... SELECT.

Table & DB Concepts

CREATE TABLE • Data Types • Temp Tables

CREATE TABLE & MySQL Data Types

Define table structure with column names, data types, and constraints.

CREATE TABLE

```
-- CREATE TABLE syntax
CREATE TABLE employees (
  emp_id      INT           NOT NULL,
  name        VARCHAR(100)  NOT NULL,
  email       VARCHAR(150),
  hire_date   DATE,
  salary      DECIMAL(10,2),
  is_active   BOOLEAN       DEFAULT TRUE,
  PRIMARY KEY (emp_id)
);

-- View table structure
DESCRIBE employees;
SHOW COLUMNS FROM employees;
```

Data Type	Use For	Example
INT	Whole numbers, IDs	emp_id INT
BIGINT	Large numbers	revenue BIGINT
DECIMAL(p,d)	Exact money values	salary DECIMAL(10,2)
FLOAT / DOUBLE	Approximate decimals	rate FLOAT
CHAR(n)	Fixed-length text	gender CHAR(1)
VARCHAR(n)	Variable-length text	name VARCHAR(100)
TEXT	Long text, descriptions	notes TEXT
DATE	Date: YYYY-MM-DD	hire_date DATE
DATETIME	Date + time	created_at DATETIME
TIMESTAMP	Auto-track time	updated_at TIMESTAMP
BOOLEAN	True / False (0 or 1)	is_active BOOLEAN
ENUM	Fixed set of values	status ENUM('A','I')

Temporary Tables in MySQL

Exist only for the current session. Auto-dropped when connection closes. Prefix: TEMPORARY.

TEMP TABLE SYNTAX

```
-- Create a temporary table
CREATE TEMPORARY TABLE
  temp_high_earners AS
SELECT emp_id, name, salary
FROM   employees
WHERE  salary > 70000;

-- Use it like a regular table
SELECT t.name,
       t.salary,
       d.dept_name
FROM   temp_high_earners t
JOIN   departments d
  ON   t.dept_id = d.dept_id;

-- Manually drop it (optional)
DROP TEMPORARY TABLE
  temp_high_earners;

-- Check if temp table exists
SHOW TABLES LIKE 'temp%';
```

Feature	Regular Table	Temp Table
Lifetime	Permanent	Current session only
Visible to	All users	Only current user
Auto-drop	Never	On session end
Storage	Disk (InnoDB)	Memory (usually)
Can use with JOINs	✅ Yes	✅ Yes
Can be indexed	✅ Yes	✅ Yes

💡 **Real Analyst Use Case:**

Store intermediate results of complex queries in a TEMPORARY TABLE, then join or filter in the next step — much cleaner than deeply nested subqueries or CTEs.

NULL Handling

IS NULL • IS NOT NULL • COALESCE • IFNULL

Handling NULL Values

NULL means 'unknown / missing value' — NOT zero, NOT empty string. Needs special handling.

⚠️ **CRITICAL: NULL ≠ 0 | NULL ≠ '' | NULL ≠ '' — NULL means UNKNOWN / MISSING**

You CANNOT use = to check for NULL: WHERE salary = NULL ← This NEVER returns rows! Always use IS NULL

NULL EXAMPLES

```
-- Find rows WITH NULL salary
SELECT * FROM employees
WHERE  salary IS NULL;

-- Find rows WITHOUT NULL
SELECT * FROM employees
WHERE  salary IS NOT NULL;

-- IFNULL: replace NULL (MySQL)
SELECT name,
       IFNULL(salary, 0) AS salary
FROM   employees;

-- COALESCE: first non-NULL value
SELECT name,
       COALESCE(bonus, incentive, 0)
            AS extra_pay
FROM   employees;

-- NULL in arithmetic → always NULL
SELECT 500 + NULL;   -- Result: NULL
```

Operation	Syntax	Result
Find NULLs	WHERE col IS NULL	Rows with NULL
Exclude NULLs	WHERE col IS NOT NULL	Rows without NULL
Replace NULL	IFNULL(col, 0)	Returns 0 if NULL
First non-NULL	COALESCE(a, b, c)	First non-NULL value
NULL in math	NULL + 5	Returns NULL
NULL = NULL	NULL = NULL	Returns FALSE
Count NULLs	COUNT(*)-COUNT(col)	NULL row count

Aggregate Functions

MAX • MIN • SUM • AVG • COUNT

Aggregate Functions

Perform calculations on multiple rows and return a SINGLE summarized value.

MAX

Highest value in column

`MAX(salary) → ₹2,50,000`

```
SELECT MAX(salary)
FROM   employees;
```

MIN

Lowest value in column

`MIN(salary) → ₹25,000`

```
SELECT MIN(salary)
FROM   employees;
```

SUM

Total sum of all values

`SUM(salary) → ₹1.2 Crore`

```
SELECT SUM(salary)
FROM   employees;
```

AVG

Arithmetic average

`AVG(salary) → ₹80,000`

```
SELECT AVG(salary)
FROM   employees;
```

COUNT

Number of rows / values

`COUNT(*) → 150 rows`

```
SELECT COUNT(*)
FROM   employees;
```

Aggregates in Action

ALL EXAMPLES

```
-- COUNT variations
SELECT COUNT(*)       AS total_rows,
       COUNT(salary)  AS with_salary,
       COUNT(DISTINCT city) AS unique_cities
FROM   employees;

-- SUM and AVG together
SELECT SUM(salary) AS total_payroll,
       AVG(salary) AS avg_salary,
       MAX(salary) AS highest,
       MIN(salary) AS lowest
FROM   employees;

-- Aggregates with WHERE
SELECT AVG(salary) AS eng_avg
FROM   employees
WHERE  department = 'Engineering';

-- Aggregate per group
SELECT   department,
         COUNT(*)    AS headcount,
         AVG(salary) AS avg_sal,
         MAX(salary) AS top_sal
FROM     employees
GROUP BY department
ORDER BY avg_sal DESC;
```

Query	Result	Notes
`COUNT(*)`	150	Counts ALL rows inc. NULLs
`COUNT(salary)`	148	Skips 2 NULL salary rows
`COUNT(DISTINCT city)`	12	Counts unique cities
`SUM(salary)`	₹1,20,00,000	Total payroll
`AVG(salary)`	₹80,000	Arithmetic mean
`MAX(salary)`	₹2,50,000	Highest earner
`MIN(salary)`	₹25,000	Lowest salary

💡 **COUNT(*) vs COUNT(col):**

COUNT(*) counts all rows including NULLs. COUNT(column) ignores NULL values in that specific column.

GROUP BY & Filtering

GROUP BY • HAVING • WHERE vs HAVING

GROUP BY Clause

Groups rows with the same value in a column. Always used with aggregate functions.

```
SELECT  col, AGG_FN(col)  FROM  table  [WHERE ...]  GROUP BY  col  [ORDER BY ...];
```

GROUP BY EXAMPLES

```
-- Count per department
SELECT   department,
         COUNT(*) AS headcount
FROM     employees
GROUP BY department;

-- Avg salary per city
SELECT   city,
         AVG(salary) AS avg_sal,
         MAX(salary) AS max_sal
FROM     employees
GROUP BY city
ORDER BY avg_sal DESC;

-- Group by multiple columns
SELECT   department,
         city,
         COUNT(*) AS staff_count
FROM     employees
GROUP BY department, city
ORDER BY department, city;
```

department	headcount	avg_salary	max_salary
Engineering	45	₹95,000	₹2,50,000
Analytics	22	₹85,000	₹1,80,000
HR	12	₹60,000	₹90,000
Finance	18	₹80,000	₹1,50,000
Marketing	20	₹70,000	₹1,20,000

💡 **RULE:**

Every column in SELECT that is NOT inside an aggregate function MUST appear in the GROUP BY clause. Otherwise MySQL throws an error.

HAVING Clause — Filtering Groups

HAVING filters GROUP BY results. WHERE filters individual rows BEFORE grouping.

HAVING EXAMPLES

```
-- HAVING: filter groups
SELECT   department,
         AVG(salary) AS avg_sal
FROM     employees
GROUP BY department
HAVING   AVG(salary) > 70000;

-- Combine WHERE + HAVING
SELECT   department,
         COUNT(*) AS cnt
FROM     employees
WHERE    status = 'Active'     ← pre-filter
GROUP BY department
HAVING   COUNT(*) >= 5         ← post-filter
ORDER BY cnt DESC;

-- HAVING with MAX
SELECT   city, MAX(salary)
FROM     employees
GROUP BY city
HAVING   MAX(salary) > 100000;
```

Feature	WHERE	HAVING
Filters	Individual ROWS	Groups (after GROUP BY)
Works with	Any column	Aggregate functions
Position in query	Before GROUP BY	After GROUP BY
Aggregates	❌ Cannot use	✅ Must use
Runs at step	2 (early)	5 (late)
Speed	Faster	Slower

FULL ORDER

```
-- Full query with correct clause order:
SELECT dept, COUNT(*), AVG(salary)  -- 6. SELECT
FROM   employees                    -- 1. FROM
WHERE  status = 'Active'            -- 2. WHERE
GROUP BY dept                       -- 3. GROUP BY
HAVING COUNT(*) > 3                 -- 4. HAVING
ORDER BY AVG(salary) DESC;          -- 5. ORDER BY
```

SQL Joins

INNER • LEFT • RIGHT • FULL OUTER • SELF • ANTI

SQL Joins — All Types at a Glance

Joins combine rows from two tables based on a related column (usually a key).

INNER JOIN

Only rows that match in BOTH tables

No unmatched rows returned

LEFT JOIN

ALL left rows + matching right rows

Right side → NULL if no match

RIGHT JOIN

ALL right rows + matching left rows

Left side → NULL if no match

FULL OUTER

ALL rows from BOTH tables

NULLs on non-matching sides

SELF JOIN

Table joined with itself

Used for hierarchy data

LEFT ANTI JOIN

Left rows with NO match in right

Find orphaned/unmatched records

INNER JOIN

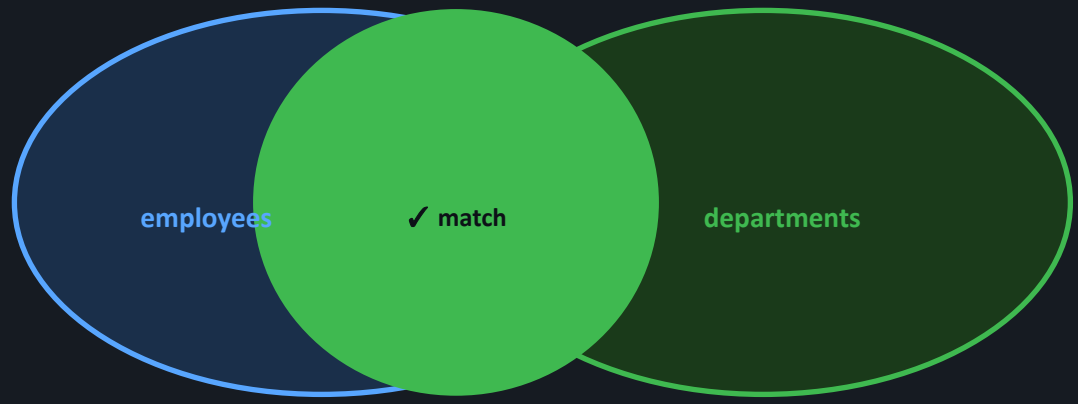
Returns ONLY rows that have a matching value in BOTH tables. Most commonly used join.

INNER JOIN

```
-- Standard INNER JOIN
SELECT e.name,
       e.salary,
       d.dept_name,
       d.location
FROM   employees  e
INNER JOIN
       departments d
  ON   e.dept_id = d.dept_id;

-- Shorthand (INNER is optional)
SELECT e.name, d.dept_name
FROM   employees e
JOIN   departments d
  ON   e.dept_id = d.dept_id;

-- Three-table join
SELECT e.name, d.dept_name, p.project_name
FROM   employees e
JOIN   departments d ON e.dept_id = d.id
JOIN   projects    p ON p.lead_id = e.emp_id;
```



Only matching rows returned

name	dept_name	location
Arjun	Engineering	Bengaluru
Priya	Analytics	Mumbai

LEFT JOIN & RIGHT JOIN

LEFT JOIN

ALL rows from LEFT + matching from right.
Non-matched right rows → NULL.

LEFT JOIN

```
SELECT e.name, d.dept_name
FROM   employees e        ← ALL
LEFT JOIN
       departments d
  ON   e.dept_id = d.dept_id;

-- LEFT ANTI JOIN
-- (employees with NO department)
SELECT e.name
FROM   employees e
LEFT JOIN departments d
  ON   e.dept_id = d.dept_id
WHERE  d.dept_id IS NULL;
```

RIGHT JOIN

ALL rows from RIGHT + matching from left.
Non-matched left rows → NULL.

RIGHT JOIN

```
SELECT e.name, d.dept_name
FROM   employees e
RIGHT JOIN
       departments d  ← ALL
  ON   e.dept_id = d.dept_id;

-- RIGHT ANTI JOIN
-- (departments with NO employees)
SELECT d.dept_name
FROM   employees e
RIGHT JOIN departments d
  ON   e.dept_id = d.dept_id
WHERE  e.emp_id IS NULL;
```

Full Outer • Self • Anti Joins

FULL OUTER

```
-- FULL OUTER JOIN
-- MySQL doesn't support it natively
-- Simulate with UNION:
SELECT e.name, d.dept_name
FROM   employees e
LEFT JOIN departments d
  ON e.dept_id = d.dept_id
UNION
SELECT e.name, d.dept_name
FROM   employees e
RIGHT JOIN departments d
  ON e.dept_id = d.dept_id;
```

SELF JOIN

```
-- SELF JOIN
-- Find employee and their manager
-- (manager_id references emp_id)
SELECT e.name  AS employee,
       m.name  AS manager
FROM   employees e
JOIN   employees m
  ON   e.manager_id = m.emp_id;

-- Employees in same department
SELECT a.name, b.name
FROM   employees a
JOIN   employees b
  ON   a.dept_id = b.dept_id
WHERE  a.emp_id < b.emp_id;
```

ANTI JOINS

```
-- LEFT ANTI JOIN
-- (find left rows with NO match)
SELECT e.name
FROM   employees e
LEFT JOIN departments d
  ON   e.dept_id = d.dept_id
WHERE  d.dept_id IS NULL;

-- RIGHT ANTI JOIN
-- (departments with NO employees)
SELECT d.dept_name
FROM   employees e
RIGHT JOIN departments d
  ON   e.dept_id = d.dept_id
WHERE  e.emp_id IS NULL;
```

Join Type	Returns	Key Trick
Full Outer	ALL rows both tables	LEFT JOIN UNION RIGHT JOIN
Self Join	Table joined to itself	Use aliases: e and m
Left Anti	Left with NO match	WHERE right.id IS NULL

Set Operations

UNION • UNION ALL

UNION and UNION ALL

Combine result sets of two SELECT queries with the same number and type of columns.

Feature	UNION	UNION ALL
Duplicate rows	REMOVED	KEPT (all rows)
Performance	Slower (deduplication step)	Faster (no dedup)
When to use	Need unique results	Need all rows including duplicates
Row count	≤ sum of both queries	= exactly sum of both queries

SYNTAX

```
-- UNION: unique cities only
SELECT city FROM customers
UNION
SELECT city FROM suppliers;

-- UNION ALL: keep all (with duplicates)
SELECT city FROM customers
UNION ALL
SELECT city FROM suppliers;

-- ORDER BY applies to FULL result
SELECT name FROM employees
UNION
SELECT name FROM contractors
ORDER BY name;
```

RULES

```
-- ⚠️ RULES for UNION:
-- Both SELECT must have SAME
-- number of columns
-- Compatible data types

-- ✅ Valid
SELECT emp_id, name FROM employees
UNION
SELECT id, full_name FROM clients;

-- ❌ Invalid (different col count)
SELECT emp_id, name, sal FROM emp
UNION
SELECT id, full_name FROM clients;
```

Conditional Logic

CASE Statement • Nested CASE

CASE Statement — SQL's IF / ELSE

Returns different values based on conditions. Like IF-ELSE in programming languages.

SYNTAX:

```
CASE  WHEN condition1 THEN result1  WHEN condition2 THEN result2  ...  ELSE default  END  [AS alias]
```

CASE EXAMPLES

```
-- Grade based on salary
SELECT name, salary,
  CASE
    WHEN salary >= 100000 THEN 'Senior'
    WHEN salary >= 70000  THEN 'Mid-Level'
    WHEN salary >= 40000  THEN 'Junior'
    ELSE                       'Intern'
  END AS grade
FROM employees;

-- Simple CASE (like switch)
SELECT name,
  CASE rating
    WHEN 'A' THEN salary * 0.20
    WHEN 'B' THEN salary * 0.10
    WHEN 'C' THEN salary * 0.05
    ELSE          0
  END AS bonus
FROM employees;
```

NESTED CASE

```
-- NESTED CASE (CASE inside CASE)
SELECT name, city, salary,
  CASE
    WHEN city = 'Mumbai' THEN
      CASE
        WHEN salary > 80000 THEN 'Mum-Senior'
        ELSE                     'Mum-Junior'
      END
    WHEN city = 'Bengaluru' THEN
      CASE
        WHEN salary > 90000 THEN 'BLR-Senior'
        ELSE                     'BLR-Junior'
      END
    ELSE 'Other'
  END AS category,
  -- CASE in ORDER BY
  CASE grade
    WHEN 'Senior' THEN 1
    WHEN 'Mid'    THEN 2
    ELSE               3
  END AS sort_key
FROM employees;
```

Advanced SQL

Subqueries • Execution Order • Comments • TOP N

Subqueries — Query Inside a Query

Inner query runs FIRST, passes result to outer query. Also called nested queries.

SUBQUERY EXAMPLES

```
-- WHERE subquery
-- Employees earning above average
SELECT name, salary
FROM   employees
WHERE  salary > (
  SELECT AVG(salary)
  FROM   employees
);

-- IN subquery
SELECT * FROM employees
WHERE  dept_id IN (
  SELECT id
  FROM   departments
  WHERE  location = 'South'
);

-- FROM subquery (derived table)
SELECT dept, avg_sal
FROM (
  SELECT   department AS dept,
           AVG(salary) AS avg_sal
  FROM     employees
  GROUP BY department
) AS dept_avg
WHERE avg_sal > 70000;
```

Type	Returns	Used In
Scalar	1 row, 1 col	WHERE, SELECT, HAVING
Column	1 col, many rows	IN / NOT IN
Row	1 row, many cols	WHERE row = (...)
Table	Multiple rows+cols	FROM clause

CORRELATED

```
-- Correlated subquery
-- (references outer query column)
SELECT e.name, e.salary, e.dept_id
FROM   employees e
WHERE  e.salary = (
  SELECT MAX(salary)
  FROM   employees
  WHERE  dept_id = e.dept_id
);
-- Finds top earner in each dept
```

MySQL Order of Execution

SQL does NOT run clauses in the order you write them. Knowing this prevents many errors.

1 FROM / JOIN

Load and combine tables

2 WHERE

Filter individual rows

3 GROUP BY

Group the filtered rows

4 HAVING

Filter the groups

5 SELECT

Pick columns, calculate expressions

6 DISTINCT

Remove duplicate rows

7 ORDER BY

Sort the final result

8 LIMIT

Return top N rows

COMPLETE EXAMPLE

```
SELECT dept, COUNT(*), AVG(sal)  -- 5.SELECT     FROM employees          -- 1.FROM
WHERE status='Active'            -- 2.WHERE       GROUP BY dept           -- 3.GROUP BY
HAVING COUNT(*)>3                -- 4.HAVING      ORDER BY AVG(sal) DESC -- 7.ORDER BY
LIMIT 10;                        -- 8.LIMIT
```

SQL Comments & TOP N Queries

SQL Comments

Add explanations to your SQL code. MySQL supports two styles.

COMMENT SYNTAX

```
-- Single-line comment (most common)
SELECT * FROM employees;

-- Another single-line comment
SELECT name, salary  -- get two columns
FROM   employees
WHERE  salary > 50000;  -- high earners

/* Multi-line comment
   This query calculates the
   department summary for Q3.
   Author: Mohan KTK
   Date: 2024-08-15
*/
SELECT   department, AVG(salary)
FROM     employees
GROUP BY department;
```

TOP N Queries

Get top/bottom N rows. MySQL uses LIMIT. No TOP keyword in MySQL.

TOP N

```
-- Top 5 highest salaries
SELECT * FROM employees
ORDER BY salary DESC
LIMIT  5;

-- Bottom 5 (lowest salaries)
SELECT * FROM employees
ORDER BY salary ASC
LIMIT  5;

-- 2nd Highest salary
SELECT MAX(salary)
FROM   employees
WHERE  salary < (
  SELECT MAX(salary)
  FROM   employees
);

-- Nth highest (using LIMIT + OFFSET)
SELECT DISTINCT salary
FROM   employees
ORDER BY salary DESC
LIMIT  1 OFFSET 2;  -- 3rd highest
```

Constraints

NOT NULL • UNIQUE • CHECK • DEFAULT • PK • FK

Column Constraints — Part 1

Rules enforced by MySQL to ensure data quality. Defined at column or table level.

NOT NULL

Column CANNOT have NULL values.
Every insert MUST provide a value.

```
name VARCHAR(100) NOT NULL
```

UNIQUE

All values in column must be different.
Allows one NULL value in MySQL.

```
email VARCHAR(150) UNIQUE
```

CHECK

Values must satisfy a condition.
MySQL enforces this from v8.0.16+.

```
age INT CHECK (age BETWEEN 18 AND 65)
```

DEFAULT

Auto-fills value if none is provided.
Used when INSERT skips the column.

```
status VARCHAR(10) DEFAULT 'Active'
```

PRIMARY KEY & FOREIGN KEY

PRIMARY KEY

Uniquely identifies every row. Automatically enforces NOT NULL + UNIQUE. One per table.

PRIMARY KEY

```
-- Column level
CREATE TABLE employees (
  emp_id INT PRIMARY KEY,
  name   VARCHAR(100)
);

-- Table level (composite PK)
CREATE TABLE order_items (
  order_id   INT,
  product_id INT,
  quantity   INT,
  PRIMARY KEY (order_id, product_id)
);

-- Add to existing table
ALTER TABLE employees
ADD PRIMARY KEY (emp_id);
```

FOREIGN KEY

Links a column to the PRIMARY KEY of another table. Enforces referential integrity.

FOREIGN KEY

```
CREATE TABLE orders (
  order_id    INT PRIMARY KEY,
  customer_id INT NOT NULL,
  amount      DECIMAL(10,2),

  -- FK: must exist in customers
  FOREIGN KEY (customer_id)
    REFERENCES customers(id)
    ON DELETE CASCADE
    ON UPDATE CASCADE
);

-- Add FK to existing table
ALTER TABLE orders
ADD FOREIGN KEY (customer_id)
  REFERENCES customers(id);
```

All 6 Constraints — One Complete Example

This is the CREATE TABLE you'll write in real MySQL projects — all constraints included.

FULL CREATE TABLE

```
CREATE TABLE employees (

  -- PRIMARY KEY: unique row identifier
  emp_id      INT           PRIMARY KEY
                            AUTO_INCREMENT,

  -- NOT NULL: required fields
  name        VARCHAR(100)  NOT NULL,
  department  VARCHAR(50)   NOT NULL,

  -- UNIQUE: no duplicate values
  email       VARCHAR(150)  UNIQUE,
  pan_number  CHAR(10)      UNIQUE,

  -- CHECK: data validation rules
  age         INT           CHECK (age BETWEEN 18 AND 65),
  salary      DECIMAL(10,2) CHECK (salary > 0),

  -- DEFAULT: auto-fill if not provided
  status      VARCHAR(10)   DEFAULT 'Active',
  hire_date   DATE          DEFAULT (CURRENT_DATE),

  -- FOREIGN KEY: referential integrity
  dept_id     INT,
  FOREIGN KEY (dept_id)
    REFERENCES departments(id)
    ON DELETE SET NULL
    ON UPDATE CASCADE
);
```

Constraint	Enforces	NULL allowed?
NOT NULL	Value must be provided	❌ No
UNIQUE	No duplicate values	✅ One NULL
CHECK	Value passes condition	✅ Yes
DEFAULT	Auto-fill if missing	N/A
PRIMARY KEY	Unique row identifier	❌ Never
FOREIGN KEY	Must exist in parent	✅ Unless NOT NULL

💡 **AUTO_INCREMENT:**
MySQL-specific feature — auto-generates unique INT values for PRIMARY KEY (1, 2, 3…). Use with INT or BIGINT PK columns.

Deletion Operations

DELETE • DROP • TRUNCATE — Know the Difference!

DELETE vs TRUNCATE vs DROP

Three different commands. Each does something very different. #1 SQL interview question!

DELETE

Removes specific rows (or all rows)

✅ Table structure stays

✅ Can be rolled back

Slow (logs each row)

✅ Supports WHERE

```
-- Delete specific rows
DELETE FROM employees
WHERE  status = 'Inactive';

-- Delete all rows (rollback-able)
DELETE FROM temp_logs;
```

TRUNCATE

Removes ALL rows instantly

✅ Table structure stays

❌ Cannot be rolled back

Very fast

❌ No WHERE clause

```
-- Remove all rows fast
TRUNCATE TABLE employees;

-- Cannot use WHERE:
-- TRUNCATE TABLE employees
-- WHERE dept = 'HR'; ← ERROR
```

DROP

Deletes the entire table

❌ Table is GONE

❌ Cannot be rolled back

Instant

❌ N/A

```
-- Remove entire table
DROP TABLE employees;

-- Drop only if exists
DROP TABLE IF EXISTS temp_data;

-- Drop entire database
DROP DATABASE company_db;
```

DELETE vs TRUNCATE vs DROP — Full Comparison

The most asked SQL interview question. Know every row of this comparison table.

Feature	DELETE	TRUNCATE	DROP
Command type	DML	DDL	DDL
What is removed	Specific/all rows	ALL rows	Entire table + structure
Table structure	Kept intact	Kept intact	Completely removed
WHERE clause	✅ Supported	❌ Not supported	❌ Not applicable
Rollback possible	✅ Yes (in transaction)	❌ No	❌ No
Performance	Slow (row-by-row)	Fast (page drop)	Instant
AUTO_INCREMENT	Not reset	Reset to 1	Table gone
Triggers fired	✅ Yes	❌ No	❌ No
Disk space freed	No	Yes	Yes

QUICK SUMMARY

```
DELETE FROM t WHERE ...;  -- ← Specific rows. DML. Rollback OK.   Slow.    Triggers fire.   Structure stays.
TRUNCATE TABLE t;         -- ← All rows.       DDL. No rollback.  Fast.    Triggers skip.   Structure stays.   AUTO_INC reset.
DROP TABLE t;             -- ← ENTIRE TABLE.   DDL. No rollback.  Instant. Everything gone.
```

SQL Joins — Quick Reference

Join	Returns	NULLs?	Best Use Case
INNER JOIN	Matching rows only	No	Standard data retrieval
LEFT JOIN	All left + matched right	Right → NULL	Find orphaned left rows
RIGHT JOIN	All right + matched left	Left → NULL	Find orphaned right rows
FULL OUTER	All rows from both	Both sides → NULL	All unmatched rows (use UNION)
SELF JOIN	Table joined to itself	Depends	Manager/employee hierarchy
LEFT ANTI	Left rows with NO match	Right → NULL	Missing / unlinked records

JOIN TEMPLATE

```
SELECT t1.col, t2.col          -- INNER:      JOIN
FROM   table1 t1                -- LEFT:   LEFT JOIN
[JOIN_TYPE]                     -- RIGHT: RIGHT JOIN
       table2 t2                -- FULL:   LEFT JOIN ... UNION ... RIGHT JOIN
  ON   t1.id = t2.id;           -- ANTI:   LEFT JOIN ... WHERE t2.id IS NULL
```

SQL Operators — Quick Reference

Operator	Category	What it does	Example
AND	Logical	Both conditions must be true	WHERE sal>50000 AND city='Hyd'
OR	Logical	Either condition can be true	WHERE city='Mum' OR city='Del'
NOT	Logical	Negates the condition	WHERE NOT dept='HR'
BETWEEN	Range	Inclusive range check	WHERE salary BETWEEN 40000 AND 80000
IN	List	Value matches list item	WHERE city IN ('Mum','Pune')
NOT IN	List	Value not in list	WHERE dept NOT IN ('HR','Legal')
LIKE	Pattern	Pattern with % and _	WHERE name LIKE 'A%'
NOT LIKE	Pattern	Does not match pattern	WHERE email NOT LIKE '%test%'
IS NULL	NULL check	Value is NULL	WHERE salary IS NULL
IS NOT NULL	NULL check	Value is not NULL	WHERE bonus IS NOT NULL

Aggregates & GROUP BY — Quick Reference

Function	Returns	NULL handling
`COUNT(*)`	Total rows including NULLs	Counts NULL rows
`COUNT(col)`	Non-NULL rows in column	Skips NULL values
`SUM(col)`	Total sum	Ignores NULLs
`AVG(col)`	Sum ÷ non-NULL count	Ignores NULLs
`MAX(col)`	Highest value	Ignores NULLs
`MIN(col)`	Lowest value	Ignores NULLs

COMPLETE TEMPLATE

```
-- Full GROUP BY query with all clauses (in WRITING order)
SELECT   department,                COUNT(*) AS headcount,  AVG(salary) AS avg_sal
FROM     employees                  -- 1. FROM
WHERE    status = 'Active'          -- 2. WHERE  → filters rows BEFORE grouping
GROUP BY department                 -- 3. GROUP BY
HAVING   COUNT(*) > 5               -- 4. HAVING → filters groups AFTER aggregation
ORDER BY avg_sal DESC               -- 5. ORDER BY
LIMIT    10;                        -- 6. LIMIT
```

All 6 Constraints — Quick Reference

Constraint	Enforces	NULL?	Key Note
NOT NULL	Column must have a value	❌ Never	Most basic constraint
UNIQUE	No duplicate values in column	✅ One NULL	email, PAN are good candidates
CHECK	Value must pass condition	✅ Yes	Enforced from MySQL 8.0.16+
DEFAULT	Auto-fill if not provided	N/A	DEFAULT 'Active', DEFAULT NOW()
PRIMARY KEY	Unique row identifier	❌ Never	One per table. Implies NOT NULL + UNIQUE
FOREIGN KEY	Must exist in parent table	✅ If not NOT NULL	ON DELETE CASCADE / SET NULL / RESTRICT

ALL CONSTRAINTS EXAMPLE

```
CREATE TABLE demo (
  id      INT           PRIMARY KEY AUTO_INCREMENT,   -- PK
  name    VARCHAR(100)  NOT NULL,                     -- NOT NULL
  email   VARCHAR(150)  UNIQUE,                       -- UNIQUE
  age     INT           CHECK (age BETWEEN 18 AND 65), -- CHECK
  status  VARCHAR(10)   DEFAULT 'Active',             -- DEFAULT
  dept_id INT           REFERENCES departments(id)    -- FK (shorthand)
);
```

DML Quick Reference

Command	Action	Safe without WHERE?
INSERT	Add new row(s)	✅ Yes — no WHERE needed
UPDATE	Modify existing rows	❌ No — updates ALL rows without WHERE
DELETE	Remove rows	❌ No — deletes ALL rows without WHERE

DML QUICK REFERENCE

```
-- INSERT (all columns — values in column order)
INSERT INTO employees VALUES (105, 'Vikram', 'Chennai', 68000);

-- INSERT (specific columns — safer and more readable)
INSERT INTO employees (emp_id, name, salary) VALUES (106, 'Anita', 75000);

-- UPDATE with WHERE (always include it!)
UPDATE employees SET salary = salary * 1.10, city = 'Pune' WHERE emp_id = 105;

-- DELETE with WHERE (always include it!)
DELETE FROM employees WHERE status = 'Inactive' AND last_login < '2023-01-01';

-- COPY: create backup table from existing
CREATE TABLE emp_backup AS SELECT * FROM employees WHERE hire_date > '2023-01-01';
```

SELECT & Filtering — Quick Reference

SELECT PATTERNS

```
-- All columns
SELECT * FROM employees;

-- Specific columns
SELECT name, salary FROM employees;

-- Column alias
SELECT salary AS monthly_pay FROM employees;

-- DISTINCT: unique values
SELECT DISTINCT city FROM employees;

-- COUNT distinct
SELECT COUNT(DISTINCT city) FROM employees;

-- ORDER BY
SELECT * FROM employees
ORDER BY salary DESC;

-- TOP 5 (MySQL uses LIMIT)
SELECT * FROM employees
ORDER BY salary DESC LIMIT 5;

-- LIMIT with OFFSET (pagination)
SELECT * FROM employees
ORDER BY emp_id LIMIT 10 OFFSET 20;
-- (page 3, 10 rows per page)
```

WHERE PATTERNS

```
-- WHERE with operators
SELECT * FROM employees
WHERE  salary BETWEEN 40000 AND 80000;

-- WHERE with LIKE
SELECT * FROM employees
WHERE  name LIKE 'A%'
   AND email LIKE '%@gmail%';

-- WHERE with IN
SELECT * FROM employees
WHERE  city IN ('Mumbai','Pune','Goa')
  AND  status != 'Resigned';

-- WHERE with NULL
SELECT * FROM employees
WHERE  bonus IS NULL;

-- WHERE with date
SELECT * FROM orders
WHERE  order_date >= '2024-01-01'
  AND  order_date < '2025-01-01';

-- Combine everything
SELECT name, salary, city
FROM   employees
WHERE  dept_id IN (1,2,3)
  AND  salary > 50000
  AND  name NOT LIKE '%Test%'
```

CASE & Subqueries — Quick Reference

CASE PATTERNS

```
-- Simple CASE
SELECT name,
  CASE
    WHEN salary >= 100000 THEN 'Senior'
    WHEN salary >= 70000  THEN 'Mid'
    WHEN salary >= 40000  THEN 'Junior'
    ELSE                       'Intern'
  END AS grade
FROM employees;

-- CASE in WHERE (rare but valid)
SELECT * FROM employees
WHERE (
  CASE department
    WHEN 'Engineering' THEN salary > 90000
    WHEN 'HR'          THEN salary > 50000
    ELSE                    salary > 60000
  END
);

-- CASE for conditional count
SELECT
  COUNT(CASE WHEN city='Mumbai' THEN 1 END) AS mum_count,
  COUNT(CASE WHEN city='Pune'   THEN 1 END) AS pune_count
FROM employees;
```

SUBQUERY PATTERNS

```
-- Scalar subquery in WHERE
SELECT * FROM employees
WHERE  salary > (SELECT AVG(salary) FROM employees);

-- IN subquery
SELECT * FROM employees
WHERE  dept_id IN (
  SELECT id FROM departments
  WHERE  location = 'South'
);

-- NOT IN subquery
SELECT * FROM employees
WHERE  emp_id NOT IN (
  SELECT emp_id FROM resigned_list
);

-- FROM subquery (derived table)
SELECT dept, avg_sal
FROM  (
  SELECT   department AS dept,
           AVG(salary) AS avg_sal
  FROM     employees
  GROUP BY department
) AS dept_avg
WHERE avg_sal > 70000;

-- EXISTS subquery
```

NULL Handling & Temp Tables — Quick Reference

NULL PATTERNS

```
-- Check for NULL
SELECT * FROM employees WHERE salary IS NULL;
SELECT * FROM employees WHERE salary IS NOT NULL;

-- Replace NULL (MySQL: IFNULL)
SELECT name, IFNULL(salary, 0) AS salary
FROM   employees;

-- Replace NULL (ANSI: COALESCE — first non-NULL)
SELECT name, COALESCE(bonus, incentive, 0) AS extra
FROM   employees;

-- NULLIF: returns NULL if both values equal
SELECT NULLIF(division, 0);  -- avoid divide-by-zero

-- NULL in arithmetic always returns NULL
SELECT 100 + NULL;  -- → NULL
SELECT name FROM employees WHERE '' IS NULL;  -- no rows
```

NULL Function	MySQL	Returns
IFNULL(col, val)	MySQL only	val if col is NULL
COALESCE(a,b,c)	ANSI standard	First non-NULL value
NULLIF(a, b)	ANSI standard	NULL if a = b, else a
IS NULL	ANSI standard	TRUE if value is NULL
IS NOT NULL	ANSI standard	TRUE if value is not NULL

TEMP TABLE PATTERN

```
-- Temp table create and use
CREATE TEMPORARY TABLE temp_big_earners AS
SELECT emp_id, name, salary FROM employees WHERE salary > 80000;

SELECT t.name, d.dept_name FROM temp_big_earners t JOIN departments d ON t.dept_id = d.id;
DROP TEMPORARY TABLE temp_big_earners;  -- optional: auto-drops on session end
```

MySQL Master Syntax Cheat Sheet

QUERY TEMPLATES

```
-- SELECT template
SELECT [DISTINCT] col1, col2, AGG(col)
FROM   table
[JOIN  table2 ON t.id = t2.id]
[WHERE  condition]
[GROUP BY col]
[HAVING AGG(col) > val]
[ORDER BY col [DESC]]
[LIMIT  n [OFFSET m]];

-- INSERT
INSERT INTO t (c1,c2) VALUES (v1,v2);
INSERT INTO t SELECT * FROM t2;

-- UPDATE
UPDATE t SET c1=v1, c2=v2 WHERE cond;

-- DELETE
DELETE FROM t WHERE cond;

-- CREATE TABLE
CREATE TABLE t (
  id   INT  PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(100) NOT NULL,
  age  INT  CHECK (age > 0),
  city VARCHAR(50) DEFAULT 'Unknown',
  FOREIGN KEY (dept_id) REFERENCES dept(id)
);
```

ALL PATTERNS

```
-- CASE
SELECT CASE WHEN c THEN v ELSE d END AS alias;

-- SUBQUERY
SELECT * FROM t WHERE col > (SELECT AVG(col) FROM t);

-- JOINS
SELECT a.col, b.col FROM a
JOIN  b ON a.id = b.id;        -- INNER
LEFT  JOIN b ON a.id = b.id;   -- LEFT
RIGHT JOIN b ON a.id = b.id;   -- RIGHT

-- UNION
SELECT col FROM t1 UNION SELECT col FROM t2;

-- GROUP BY full template
SELECT col, COUNT(*) FROM t
WHERE  cond
GROUP BY col
HAVING COUNT(*) > n
ORDER BY col DESC;

-- DELETE vs TRUNCATE vs DROP
DELETE FROM t WHERE cond;   -- DML, rollback OK
TRUNCATE TABLE t;           -- DDL, fast, no rollback
DROP TABLE t;               -- DDL, removes table

-- NULL functions
IFNULL(col, 0)    COALESCE(a,b,c)    IS NULL
```

Top 10 SQL Interview Questions

These exact questions appear in 90%+ of Data Analyst interviews. Know all answers.

Q1
DELETE vs TRUNCATE vs DROP?

DELETE removes rows (rollback OK). TRUNCATE removes all rows fast (no rollback, resets AUTO_INCREMENT). DROP removes the entire table.

Q2
WHERE vs HAVING?

WHERE filters rows BEFORE GROUP BY. HAVING filters groups AFTER. You cannot use aggregate functions in WHERE.

Q3
INNER JOIN vs LEFT JOIN?

INNER returns only matching rows. LEFT returns ALL left rows + NULLs for unmatched right.

Q4
What is PRIMARY KEY?

A column (or set) that uniquely identifies each row. Automatically enforces NOT NULL + UNIQUE. One per table.

Q5
What is FOREIGN KEY?

A column referencing another table's PRIMARY KEY. Enforces referential integrity between tables.

Q6
COUNT(*) vs COUNT(col)?

COUNT(*) counts ALL rows including NULLs. COUNT(col) skips NULL values in that column.

Q7
SQL Order of Execution?

FROM → JOIN → WHERE → GROUP BY → HAVING → SELECT → DISTINCT → ORDER BY → LIMIT

Q8
What is a Subquery?

A SELECT nested inside another query. Inner query runs first, passes result to outer query.

Q9
UNION vs UNION ALL?

UNION removes duplicates (slower). UNION ALL keeps all rows including duplicates (faster).

Q10
What is DISTINCT?

Removes duplicate rows from result. Applied after all clauses except ORDER BY and LIMIT.

Useful MySQL Built-in Functions

Function	Category	What it Does	Example
`NOW()`	Date/Time	Current date and time	SELECT NOW()
`CURDATE()`	Date	Current date only	WHERE date = CURDATE()
`YEAR(date)`	Date	Extracts year	YEAR(hire_date) = 2024
`DATEDIFF(a,b)`	Date	Days between two dates	DATEDIFF(NOW(), hire_date)
`UPPER(s)`	String	Convert to UPPERCASE	UPPER(name)
`LOWER(s)`	String	Convert to lowercase	LOWER(email)
`CONCAT(a,b)`	String	Join strings together	CONCAT(first,' ',last)
`LENGTH(s)`	String	String length in bytes	LENGTH(name)
`ROUND(n,d)`	Math	Round to d decimals	ROUND(salary/12, 2)
`ABS(n)`	Math	Absolute (positive) value	ABS(balance)
`IFNULL(v,d)`	NULL	Replace NULL with default	IFNULL(bonus, 0)
`COALESCE(...)`	NULL	First non-NULL value	COALESCE(a,b,c,0)

MySQL Administrative Commands

Essential commands for exploring your MySQL environment — used constantly in real work.

SHOW/DESCRIBE

```
-- Database operations
SHOW DATABASES;
CREATE DATABASE company_db;
USE   company_db;
DROP  DATABASE IF EXISTS old_db;

-- Table operations
SHOW TABLES;
SHOW TABLES LIKE 'emp%';

-- Table structure
DESCRIBE employees;
DESC  employees;      -- shorthand
SHOW COLUMNS FROM employees;

-- Create statement of a table
SHOW CREATE TABLE employees;

-- Table info (size, rows, etc.)
SHOW TABLE STATUS LIKE 'employees';

-- Current database
SELECT DATABASE();
```

ALTER TABLE

```
-- Column operations
ALTER TABLE employees
ADD    COLUMN phone VARCHAR(15);

ALTER TABLE employees
MODIFY COLUMN salary DECIMAL(12,2);

ALTER TABLE employees
CHANGE COLUMN name full_name VARCHAR(150);

ALTER TABLE employees
DROP   COLUMN phone;

-- Rename table
RENAME TABLE employees TO staff;
-- or:
ALTER TABLE old_name RENAME TO new_name;

-- Add/Drop index
CREATE INDEX idx_city ON employees(city);
DROP   INDEX idx_city ON employees;

-- View indexes
SHOW INDEX FROM employees;
```

COURSE COMPLETE

All 13 SQL Topics Covered!

1. Basic Queries
2. Operators
5. NULL
4. Tables/DB
6. Aggregates
7. GROUP BY
10. CASE
9. Set Ops
11. Advanced
12. Constraints
13. Deletions

🚀 **NEXT STEPS TO MASTER SQL:**

1. Practice each query type on a real dataset (Kaggle) 2. Build a portfolio project (Employee / Sales analysis) 3. Solve LeetCode SQL Easy → Medium 4. Revise the 10 interview questions daily